TECHNICAL PRODUCT REQUIREMENT DOCUMENT (PRD) & SYSTEM ARCHITECTURE SPECIFICATION

Project Code Name: Hedgify
Version: 1.0.0-FINAL
Target Platform: Alpaca Markets API / MCP Agent Environment
Event: Alpaca AI Trading Agents Hackathon (lablab.ai)
Timeline: 7-Day Build (Aug 28 – Sep 4, 2026)
Environment: Paper Trading Only

---

1. EXECUTIVE SUMMARY & CORE OBJECTIVES

1.1 Problem Statement

Retail investors holding equity portfolios face an asymmetric risk dilemma: unexpected market shocks or earnings-driven volatility can trigger rapid, unrecoverable drawdowns. Standard stop-loss mechanisms force complete liquidation of underlying assets, crystallizing losses, triggering tax liabilities, and causing investors to miss potential rebounds. The core financial pain point is the absence of **real-time automated downside protection that preserves capital without selling the underlying portfolio**.

Execution risks include:

- Human emotional bias (panic-selling, revenge trading)
- Latency in manual hedging decisions during market crashes
- Complexity barrier preventing retail adoption of institutional-grade options strategies

1.2 Proposed Idea & Execution

Hedgify is an autonomous multi-agent risk management system that continuously monitors portfolio health and automatically executes protective put options via the Alpaca brokerage platform when predefined drawdown thresholds are breached. Instead of selling assets, the system purchases insurance (put options) to mathematically cap downside loss while preserving full upside exposure.

The system employs a **Supervisor-Worker multi-agent architecture** connected through the **Model Context Protocol (MCP)**, enabling natural-language intent translation into precise brokerage API calls. All operations execute within Alpaca's paper trading environment using real market data and fake capital.

1.3 Core Objectives & KPIs

Objective	KPI	Target
Capital Preservation	Drawdown mitigation rate	60% loss reduction vs. unhedged
System Reliability	Alert-to-order latency	< 10 seconds
Automation Coverage	Drawdown capture rate	100% (all 2% drops trigger)
Fault Tolerance	API error recovery	Auto-retry 3x, zero crash tolerance
Demo Performance	Dashboard load time	< 2 seconds
Idempotency	Duplicate order prevention	100%

2. SCOPE BOUNDARIES & SYSTEM REQUIREMENTS

2.1 In-Scope Features

ID	Feature	Description
IS-01	Paper Trading Integration	All orders execute via Alpaca paper trading API
IS-02	2-Stock Portfolio Demo	Support 2-3 equity positions for end-to-end demonstration
IS-03	Protective Put Hedging	Automatic purchase of 5% OTM put options on drawdown trigger
IS-04	2% Drawdown Trigger	Fixed threshold: hedge fires when portfolio drops 2% from peak
IS-05	14-Day Expiry Rule	Put options expire ~14 days from purchase
IS-06	2-Agent Architecture	Monitor Agent (Supervisor) + Hedge Executor (Worker)
IS-07	MCP Integration	Agent intent translated to Alpaca API via MCP Server
IS-08	SQLite State Persistence	Peak value, active hedges, alert history stored locally
IS-09	FastAPI Backend	REST API + WebSocket for frontend communication
IS-10	React Dashboard	Real-time portfolio view, alert feed, active hedges, stress test
IS-11	Synthetic Stress Testing	Manual crash simulation to validate hedge payoff
IS-12	Idempotency Guard	Prevents duplicate hedge orders on same stock
IS-13	Background Agent Loop	Monitor runs as asyncio task within FastAPI
IS-14	WebSocket Push	Live dashboard updates without polling

2.2 Out-of-Scope Explicit Boundaries

ID	Boundary	Rationale
OS-01	Real Money Trading	Safety, regulatory compliance, hackathon requirement
OS-02	Multiple Hedge Strategies	No collars, straddles, spreads, covered calls
OS-03	Live Greeks Calculation	Delta/Vega/Theta computed in real-time; use fixed 5% OTM heuristic

Table 3 – continued

ID	Boundary	Rationale
OS-04	News Sentiment / Macro Analysis	No NLP scraping, no Fed announcement parsing
OS-05	AI Price Prediction	Not predicting direction; only insuring against drops
OS-06	Multi-Broker Support	Alpaca only for hackathon demo
OS-07	Options Assignment Handling	Assume puts expire worthless or are closed before expiry
OS-08	Portfolio Rebalancing	No selling winners / buying losers
OS-09	Tax Optimization	Irrelevant for paper trading
OS-10	User Auth / Multi-User	Single-user demo system
OS-11	Years of Historical Backtesting	One controlled stress test sufficient
OS-12	Reinforcement Learning	Rule-based only; no ML training
OS-13	Crypto Options	Stock options only
OS-14	Mobile Native App	Web dashboard only

2.3 Functional Requirements

FR-01: Portfolio Monitoring The Monitor Agent shall poll the Alpaca Paper Trading API every 15 minutes to fetch current portfolio value and individual stock positions.

FR-02: Peak Value Tracking The system shall maintain and persist the highest portfolio value reached (the "peak") in SQLite.

FR-03: Drawdown Detection The Monitor Agent shall calculate percentage drop from peak. If drop 2%, fire a `DRAWDOWN_EVENT` containing stock symbol and severity.

FR-04: Event Handoff The Monitor Agent (Supervisor) shall pass the `DRAWDOWN_EVENT` to the Hedge Executor Agent (Worker) via a clean, structured JSON message with no shared memory.

FR-05: Put Option Selection The Hedge Executor Agent shall calculate put option parameters using the fixed rule:

- Strike price = 95% of current stock price (5% OTM)
- Expiration = ~14 days from trigger date
- Quantity = 1 contract per 100 shares held

FR-06: MCP Intent Translation The Hedge Executor Agent shall send a natural-language intent (e.g., "Buy protective put for AAPL, strike $190, expiry Sept 12") to the Alpaca MCP Server.

FR-07: Order Execution The MCP Server shall translate intent into an Alpaca Options API call and execute the paper trade, returning an order confirmation.

FR-08: Idempotency Guard The Hedge Executor Agent shall check SQLite before placing an order. If an active put already exists for that stock, skip the trade.

FR-09: State Persistence The system shall log every event, alert, and order to SQLite with timestamps for auditability and dashboard display.

FR-10: Live Dashboard The Streamlit dashboard shall display: current portfolio value and peak value, active hedges (open put options), alert history with timestamps, simulated stress-test results.

FR-11: Synthetic Stress Testing The system shall allow manual input of a hypothetical stock price drop (e.g., -15%) and calculate P&L impact with and without the protective put.

FR-12: Error Handling & Retry If an Alpaca API call fails, the agent shall log the error, wait 30 seconds, and retry up to 3 times before marking the event as failed.

2.4 Non-Functional Requirements

NFR-01: Fault Tolerance The system shall not crash if Alpaca's API is temporarily unavailable. Failed calls must be logged and retried gracefully without corrupting portfolio state.

NFR-02: State Recoverability If the system restarts, it must resume from its last known state — peak value, active hedges, and pending alerts — with zero data loss from SQLite.

NFR-03: API Key Security Alpaca API keys must be stored in environment variables (`.env`), never hardcoded or committed to version control.

NFR-04: Demo Performance The React dashboard must load in under 3 seconds and update within 5 seconds of a new event.

NFR-05: Modular Codebase Each layer (Monitor, Executor, MCP bridge, Dashboard) must be in separate, well-named Python/TypeScript modules.

NFR-06: Comprehensive Logging Every agent action (poll, alert, intent, order, error) must be written to a timestamped log file.

NFR-07: Portability The entire system must run on a single machine with only Python and Node.js installed — no Docker, no cloud servers.

NFR-08: Idempotency The same drawdown event must never trigger duplicate hedge orders, even if the Monitor polls twice before the Executor confirms.

NFR-09: Graceful Degradation If the MCP server has a bug, the system must fall back to direct Alpaca API calls so the demo still works.

NFR-10: Documentation Every module must have a 2-line docstring. The README must explain setup, running, and the demo scenario in under 5 minutes.

---

3. MATHEMATICAL FINANCE & QUANTITATIVE STRATEGY ENGINE

3.1 Financial Foundations & Terminology

Term	Definition
Protective Put	A hedging strategy where a put option is purchased for an owned stock, giving the holder the right to sell at the strike price, capping downside.
Out-of-The-Money (OTM)	A put option whose strike price is below the current market price of the underlying asset.

Table 4 – continued

Term	Definition
Strike Price (K)	The predetermined price at which the put option holder can sell the underlying stock.
Drawdown	The decline from a historical peak in portfolio value during a specific period.
Implied Volatility (IV)	The market' s forecast of a likely movement in a security' s price, derived from option prices.
Greeks	Sensitivity measures of option prices to various factors (Delta, Vega, Theta, Gamma).
Hedge Ratio	The number of options contracts required to offset the risk of the underlying position.

3.2 Protective Put Hedging Engine

The core strategy is the **Protective Put**, an institutional-grade hedging technique.

Payoff at Expiry:

$$Payoff_{stock} = S_T - S_0$$

$$Payoff_{put} = max(K - S_T, 0) - P_{premium}$$

$$Payoff_{total} = (S_T - S_0) + max(K - S_T, 0) - P_{premium}$$

Where:

- S_T = Stock price at expiry
- S_0 = Stock price at purchase
- K = Strike price of put option
- $P_{premium}$ = Premium paid for the put option

Maximum Loss (Floor):

$$MaxLoss = (S_0 - K) + P_{premium}$$

Hedge Selection Rule (Simplified — No Live Greeks):

Given the 7-day hackathon constraint, the system uses a **fixed heuristic** instead of real-time Black-Scholes computation:

$$K = S_{current} \times (1 - \alpha)$$

Where:

- $\alpha = 0.05$ (5% OTM buffer — configurable as `OTM_BUFFER = 0.05`)
- $T_{expiry} = 14$ days from trigger date
- $Q = \lfloor \frac{Shares_{held}}{100} \rfloor$ (1 contract = 100 shares)

Why this is valid: Fixed-percentage OTM protective puts are a recognized heuristic used by robo-advisors and institutional risk desks when real-time IV data is unavailable or when speed of execution outweighs precision.

3.3 Dynamic Drawdown Detection

Real-Time Drawdown Formula:

$$Drawdown_t = \frac{Peak_t - PortfolioValue_t}{Peak_t}$$

Where:

- $Peak_t = \max(PortfolioValue_0, PortfolioValue_1, ..., PortfolioValue_t)$
- $PortfolioValue_t = \sum_{i=1}^{n} (Shares_i \times Price_{i,t})$

Trigger Condition:

$$IF\ Drawdown_t \geq \theta\ THEN\ FireEvent(DRAWDOWN)$$

Where:

- $\theta = 0.02$ (2% threshold — configurable as `DRAWDOWN_THRESHOLD = 0.02`)

Peak Update Rule:

$$Peak_t = \begin{cases} PortfolioValue_t & \text{if } PortfolioValue_t > Peak_{t-1} \\ Peak_{t-1} & \text{otherwise} \end{cases}$$

3.4 Synthetic Stress Testing Engine

Since a real market crash cannot be guaranteed during the hackathon, the system includes a **controlled stress test** to validate hedge efficacy.

Stress Test Input:

- User specifies hypothetical price drop δ for a given stock (e.g., $\delta = -0.15$ for -15%)

Unhedged P&L:

$$P\&L_{unhedged} = Shares \times S_0 \times \delta$$

Hedged P&L:

$$P\&L_{hedged} = Shares \times S_0 \times \delta + Q \times 100 \times max(K - S_0(1+\delta), 0) - Q \times 100 \times P_{premium}$$

Capital Preservation Rate:

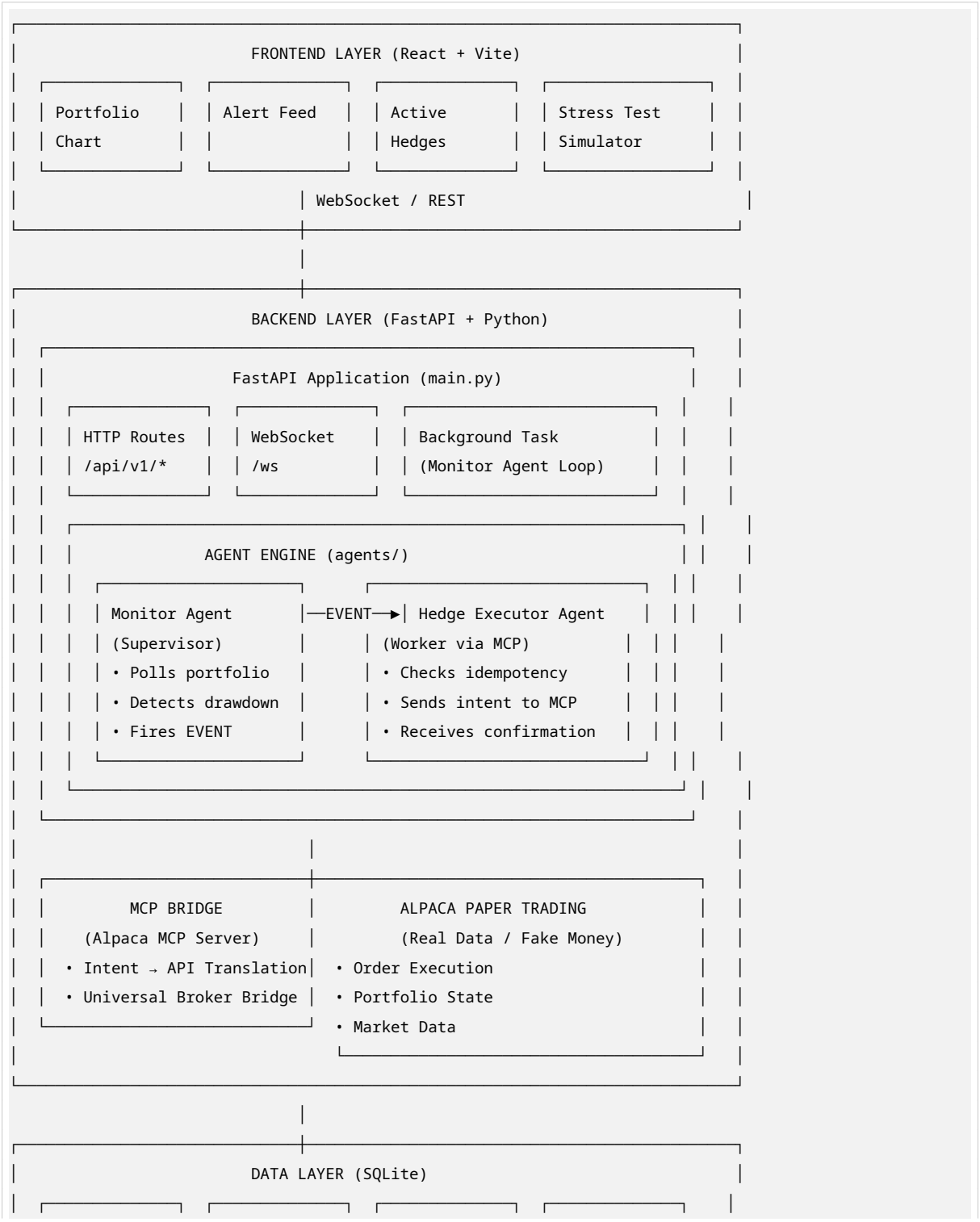
$$CPR = \frac{P\&L_{unhedged} - P\&L_{hedged}}{|P\&L_{unhedged}|} \times 100\%$$

Target: $CPR \geq 60\%$ (the hedge must save at least 60% of the would-be loss).

---

4. MULTI-AGENT ARCHITECTURE & SYSTEM DESIGN

4.1 System Architecture Diagram & Data Flow



```
|  | portfolio_   |  | alerts       |  | hedges       |  | events       |    |
|  | snapshots    |  |              |  |              |  |              |    |
|  └──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘    |
└──────────────────────────────────────────────────────────────────────────┘
```

4.2 Multi-Agent Orchestration & Workflow

Agent Communication Contract (JSON):

```
{
  "event_id": "uuid-v4",
  "event_type": "DRAWDOWN",
  "timestamp": "2026-08-28T15:30:00Z",
  "payload": {
    "stock_symbol": "AAPL",
    "current_price": 200.00,
    "shares_held": 100,
    "portfolio_value": 10000.00,
    "peak_value": 10204.08,
    "drawdown_pct": 0.021,
    "trigger_threshold": 0.02
  }
}
```

State Machine — Monitor Agent:

```
[INIT] → [POLLING] ──drawdown≥2%──► [ALERT_FIRED] ──► [SLEEP_15MIN] → [POLLING]
           |                                                              |
           └─drawdown<2%──────────────────────────────────────────────────┘
```

State Machine — Hedge Executor:

```
[IDLE] ──EVENT_RECEIVED──► [VALIDATING] ──already hedged?──► [SKIP] → [IDLE]
                                |
                                └─not hedged──► [CALCULATING] ──► [MCP_INTENT]
                                                                      |
                                                                      ▼
                                                                [AWAIT_CONFIRMATION]
                                                                      |
                                                                confirmed──► [EXECUTED] → [IDLE]
                                                                failed────► [RETRY×3] → [FAILED] → [IDLE]
```

Separation of Concerns:

- Monitor Agent never trades. It only watches and alerts.
- Hedge Executor never polls. It only reacts to events.
- Both share nothing except the JSON event message (loose coupling).

4.3 Model Context Protocol (MCP) Integration

MCP Tool Schema (Agent → MCP Server):

```
mcp_tools:
  - name: "buy_protective_put"
    description: "Purchase a protective put option via Alpaca"
    parameters:
      symbol:
        type: string
        description: "Underlying stock symbol (e.g., AAPL)"
      strike_price:
        type: number
        description: "Strike price for the put option"
      expiration_date:
        type: string
        format: "YYYY-MM-DD"
        description: "Expiration date of the option"
      quantity:
        type: integer
        description: "Number of contracts (1 contract = 100 shares)"
    returns:
      order_id: string
      status: "filled" | "pending" | "rejected"
      filled_price: number
      timestamp: string
```

MCP Fallback Strategy: If MCP Server is unavailable or returns an error, the system falls back to direct Alpaca SDK calls:

```
try:
    result = mcp_client.call("buy_protective_put", params)
except MCPConnectionError:
    result = alpaca_client.submit_order(
        symbol=params["symbol"],
        qty=params["quantity"],
        side="buy",
        type="limit",
        time_in_force="day",
        order_class="simple",
        option_symbol=build_option_symbol(params)
    )
```

4.4 Evaluation Harness (Agent Evals)

Automated Test Suite:

Test Case	Input	Expected Output	Pass Criteria
TC-01: Normal Drawdown	Portfolio drops 2.1%	EVENT fired, hedge ordered	Order placed within 10s
TC-02: No Drawdown	Portfolio drops 1.5%	No action	Zero API calls to options
TC-03: Idempotency	Same stock drops 2.1% twice	First: order. Second: skip.	Exactly 1 order in DB

Table 5 – continued

Test Case	Input	Expected Output	Pass Criteria
TC-04: API Failure	Alpaca returns 500	Retry 3x, then log failure	Log contains 3 retries
TC-05: Stress Test	-15% crash on AAPL	CPR 60%	Math output verified
TC-06: State Recovery	Restart after crash	Resume from last peak	Peak value unchanged

Hallucination Guard: The Hedge Executor must validate all calculated parameters against Alpaca's options chain before placing an order. If the calculated strike/expiry combination does not exist, the agent logs the error and aborts — never guessing.

---

5. INFRASTRUCTURE, INTEGRATIONS & TECH STACK

5.1 Tech Stack Matrix

Layer	Technology	Version	Purpose
Frontend	React	18.x	UI framework
Frontend	Vite	5.x	Build tool / dev server
Frontend	TypeScript	5.x	Type safety
Frontend	Tailwind CSS	3.x	Utility-first styling
Frontend	Recharts	2.x	Portfolio visualization
Frontend	Native WebSocket	–	Real-time data push
Backend	Python	3.11+	Agent runtime
Backend	FastAPI	0.110+	REST API + WebSocket
Backend	SQLAlchemy	2.x	ORM for SQLite
Backend	Alpaca SDK	latest	Brokerage API client
Backend	MCP Client	latest	Model Context Protocol
Backend	Uvicorn	latest	ASGI server
Database	SQLite	3.x	Zero-setup persistence
DevOps	python-dotenv	latest	Environment variable management
DevOps	Loguru	latest	Structured logging

5.2 Live Brokerage API Integration (Alpaca)

REST Endpoints Used:

Endpoint	Method	Purpose
`/v2/account`	GET	Fetch portfolio value, buying power
`/v2/positions`	GET	List current stock holdings
`/v2/orders`	POST	Submit options orders
`/v2/orders/{order_id}`	GET	Check order status
`/v2/assets/{symbol}`	GET	Validate stock symbol

WebSocket Streams Used:

Stream	Data
`wss://stream.data.alpaca.markets/v2/iex`	Real-time trade updates (optional, for demo flair)

Authentication:

- Paper API Key + Secret stored in `.env`
- `APCA_API_BASE_URL=https://paper-api.alpaca.markets`
- `APCA_API_DATA_URL=https://data.alpaca.markets`

5.3 End-to-End Autonomous Pipeline

Pipeline Sequence:

```
[1] STARTUP
    └─ FastAPI boots
    └─ SQLite initializes (creates tables if not exist)
    └─ Background task spawns Monitor Agent
    └─ React dashboard connects via WebSocket

[2] MONITORING LOOP (every 15 min)
    └─ Monitor Agent → GET /v2/account + /v2/positions
    └─ Calculate PortfolioValue_t = Σ(Shares × Price)
    └─ Update Peak_t if PortfolioValue_t > Peak_{t-1}
    └─ Calculate Drawdown_t
    └─ IF Drawdown_t ≥ 0.02:
        └─ Construct DRAWDOWN_EVENT JSON
        └─ Write event to SQLite
        └─ Broadcast via WebSocket to React
        └─ Pass event to Hedge Executor

[3] HEDGE EXECUTION
    └─ Hedge Executor receives EVENT
    └─ Query SQLite: SELECT * FROM hedges WHERE symbol = ? AND status = 'active'
    └─ IF active hedge exists: SKIP, log "Already protected"
    └─ ELSE:
        └─ Calculate K = S_current × 0.95
        └─ Calculate expiry = today + 14 days
        └─ Calculate Q = floor(shares / 100)
        └─ Send MCP intent: "Buy protective put for {symbol}, strike {K}, expiry {date}, qty {Q}"
        └─ MCP translates → Alpaca API call
        └─ Alpaca returns order confirmation
        └─ Write hedge to SQLite (status: active)
        └─ Broadcast update via WebSocket

[4] DASHBOARD DISPLAY
    └─ React receives WebSocket message
    └─ Re-renders components: PortfolioChart, AlertLog, ActiveHedges
    └─ User sees: "Hedge active on AAPL. Downside capped."
```

```
[5] STRESS TEST (manual trigger)
    └─ User inputs hypothetical drop δ
    └─ FastAPI calculates P&L_unhedged and P&L_hedged
    └─ Returns CPR and breakdown to React
    └─ User sees proof of hedge efficacy
```

5.4 Real-Time Financial Dashboard & UX

Dashboard Components:

Component	Data Source	Update Method
Portfolio Value Card	SQLite latest snapshot	WebSocket push
Peak Value Card	SQLite peak record	WebSocket push
Drawdown Gauge	Computed from above	Real-time calc
Alert Feed	SQLite alerts table	WebSocket push
Active Hedges Table	SQLite hedges table	WebSocket push
Portfolio Chart (Recharts)	SQLite snapshot history	REST fetch on load
Stress Test Form	User input + SQLite	REST POST → result

Human-in-the-Loop Controls:

Control	Function
Pause Monitor	Temporarily stops the Monitor Agent polling loop
Manual Hedge Override	Allows user to manually trigger a hedge on any stock
Kill Switch	Immediately cancels all pending orders and pauses the system
Reset Peak	Resets the peak value tracker (for demo purposes)

WebSocket Message Schema:

```
{
  "type": "PORTFOLIO_UPDATE",
  "timestamp": "2026-08-28T15:30:00Z",
  "data": {
    "portfolio_value": 10000.00,
    "peak_value": 10204.08,
    "drawdown_pct": 0.021,
    "alerts": [...],
    "active_hedges": [...]
  }
}
```

APPENDIX A: FOLDER STRUCTURE

```
hedgify/
├── frontend/                          # React (Vite + Tailwind)
│   ├── src/
│   │   ├── components/
│   │   │   ├── PortfolioChart.tsx
│   │   │   ├── AlertLog.tsx
│   │   │   ├── ActiveHedges.tsx
│   │   │   ├── StressTest.tsx
│   │   │   └── DashboardHeader.tsx
│   │   ├── hooks/
│   │   │   └── useWebSocket.ts
│   │   ├── types/
│   │   │   └── index.ts
│   │   ├── App.tsx
│   │   └── main.tsx
│   ├── index.html
│   ├── package.json
│   ├── tailwind.config.js
│   └── vite.config.ts
│
├── backend/                           # FastAPI (Python)
│   ├── main.py                        # FastAPI app, WebSocket, startup
│   ├── config.py                      # Settings, env vars, constants
│   ├── api/
│   │   ├── routes.py                  # REST endpoints
│   │   └── websocket.py               # WebSocket manager
│   ├── agents/
│   │   ├── monitor.py                 # Monitor Agent class
│   │   ├── executor.py                # Hedge Executor class
│   │   └── bridge.py                  # MCP wrapper + fallback
│   ├── models/
│   │   └── database.py                # SQLAlchemy models + SQLite engine
│   ├── services/
│   │   ├── alpaca_client.py           # Alpaca SDK wrapper
│   │   └── stress_test.py             # P&L calculator
│   ├── schemas/
│   │   └── events.py                  # Pydantic event models
│   ├── tests/
│   │   └── test_harness.py            # Automated evaluation suite
│   ├── .env                           # Alpaca API keys (gitignored)
│   └── requirements.txt
│
├── README.md                          # Setup + demo instructions
└── docs/
    └── PRD.md                         # This document
```

APPENDIX B: CONFIGURABLE CONSTANTS

```
# config.py
DRAWDOWN_THRESHOLD = 0.02           # 2% trigger
OTM_BUFFER = 0.05                   # 5% OTM
DAYS_TO_EXPIRY = 14                 # Put option expiry
POLL_INTERVAL_SECONDS = 900         # 15 minutes
MAX_RETRIES = 3                     # API retry count
RETRY_DELAY_SECONDS = 30            # Retry backoff
WEBSOCKET_BROADCAST_INTERVAL = 5    # Push every 5s
STRESS_TEST_DEFAULT_DROP = -0.15    # -15% default crash
MIN_CAPITAL_PRESERVATION_RATE = 0.60 # 60% CPR target
```

APPENDIX C: USER JOURNEYS SUMMARY

Journey	Trigger	System Action	User Sees
Happy Path	Portfolio drops 2%+	Monitor fires event → Executor buys put	"Hedge active on AAPL"
No Action	Portfolio healthy	Monitor logs "All clear", sleeps	Steady green dashboard
Already Hedged	Second drop on same stock	Executor checks DB → skips	"Already protected" log
Stress Test	User clicks "Simulate Crash"	FastAPI calculates P&L with/without hedge	"You saved $1,250" chart
API Failure	Alpaca returns 500	Retry 3x → log failure → alert user	"Connection issue — retrying"
System Restart	Server reboot	SQLite restores peak + active hedges	Seamless resume

---

End of Document — Version 1.0.0-FINAL